

From netlist to verified model

Automatic real-number-model and testbench generation
for mixed-signal verification

Veylon Systems

Abstract

Real number models are how a mixed-signal block joins a digital regression suite — and writing one by hand is slow, keeping it faithful as the circuit changes is slower, and proving it is faithful is the part that usually never happens. **spice2rnm** closes the loop in one command: give it a SPICE netlist and it delivers a SystemVerilog real-number model, a UVM-MS verification environment, and the evidence that both agree with the circuit.

Three properties set it apart. Every golden value in every generated testbench is a *SPICE measurement* of your circuit, never the model's own equations. Every generated environment is proven able to fail, by injecting faults into the model and confirming the verdict flips. And the verification strategy adapts to what each block is *for* — a filter is checked as a frequency response, a duty-cycle corrector by where its edges land, a phase interpolator by phase per code.

This paper presents four case studies of increasing difficulty, with results produced end to end by the shipped tool in under six minutes on a laptop.

Contents

1	The problem	2
2	What you get from one command	2
3	Case study 1 — Two-pole filter	3
4	Case study 2 — Duty-cycle corrector	3
4.1	A model that follows the circuit's dynamics	3
4.2	Verified by function, not by formula	3
5	Case study 3 — Phase interpolator	3
6	Case study 4 — Phase-locked loop	4
6.1	Checked as a system, against the transistors	4
6.2	Where the block works is measured, not assumed	5
7	Results at a glance	6
8	Why a green testbench is worth believing	7
9	Functional specifications as input	7
10	Joining an existing model library	8
11	AI where it helps, physics where it decides	9
12	Scope and requirements	9

1 The problem

A behavioural model has to be two contradictory things: fast enough to run in a digital regression, and faithful enough that passing it means something. Hand-written models drift from their circuits, and the drift is silent — the model still compiles, still simulates, still passes.

The usual defence is a hand-written testbench, which has the same problem one level up. Somebody decides what to check and how close is close enough, and those decisions are rarely written down next to the numbers they produced. When the circuit is revised, the model, the testbench, and the tolerances all have to be revisited by hand — so in practice they are not.

spice2rnm regenerates all three from the netlist, on demand. The model, the environment that checks it, and the evidence trail are outputs of one command, so they can never fall out of step with the circuit or with each other.

2 What you get from one command

```
spice2rnm dcc.cir --input-node clk --output-node vout \
--emit-uvm-ms --spec dcc_spec.md
```

- **A SystemVerilog real-number model**, with the model structure chosen automatically to match the block’s measured behaviour — including blocks whose dynamics move with signal level, where any fixed-response model is wrong across most of its range.

Its ports are yours to choose (`--model-ports`). The computing core is emitted with plain **real** ports, because a **real** port is a *variable*: it depends on no package, no nettype declaration and no simulator extension, so it compiles standalone anywhere. Around that core the tool emits the boundary your flow uses — **wreal**, or a user-defined nettype taken from your own library (Section 10) — so the model presents the same ports as the models beside it.

The core is untouched in every case — it is the file the equivalence check ran against — and a boundary costs nothing in fidelity: driven with identical stimulus, boundary and bare core agree exactly, worst difference 0 over 120 randomised samples across all three case-study models, and again on a library’s own nettype. **wreal** is a simulator extension rather than IEEE 1800, so that boundary is a separate file the core does not depend on.

- **A UVM-MS verification environment** following the Accellera UVM-MS specification, built on IEEE 1800 — not on Verilog-AMS. Analog values travel on *user-defined nettypes* (IEEE 1800-2017 §6.6.7) with a generated resolution function, so multiple drivers on a net — an offset source, a noise injector, a second bridge — combine in the net type rather than in hand-edited testbench code; the default output contains no **wreal** and needs no AMS licence to run. Bridge and wrapper ports are **interconnect** (§6.6.8), inheriting the testbench’s nettype instead of naming one, and scoreboard golden values are SPICE measurements of your circuit.
- **Equivalence evidence**: circuit and model driven with the identical stimulus, compared in the domain that matters for the block — waveform error for signal-path blocks, edge placement for clock-path blocks — with the aligned traces archived so any verdict can be re-derived from its own run directory.

An optional plain-language functional specification supplies what no netlist can: the block’s intended operating range and its acceptance criteria (Section 9).

3 Case study 1 — Two-pole filter

A two-pole RC low-pass: the representative signal-path case. The generated model is checked four independent ways.

Chirp equivalence	0.0290 against a 0.15 threshold	PASS
PRBS (independent stimulus)	0.0097 normalised	PASS
UVM-MS, DC section	51 checks, 0 failed	
UVM-MS, AC section	19 checks, 0 failed	
worst magnitude	0.0390 dB of a 0.25 dB tolerance	
worst phase	2.70° of an 8.00° tolerance	

The PRBS row matters more than it looks: the model is fitted against a chirp and then checked against a pseudo-random sequence it never saw. A model that had merely memorised its characterisation stimulus would be exposed here.

4 Case study 2 — Duty-cycle corrector

A comparator-based duty-cycle corrector whose dominant pole moves by a factor of 613,666 across its input range. Blocks like this defeat fixed-response modelling outright, and they defeat conventional AC-based verification too — which is exactly why they are usually the blocks whose hand-written models are trusted least.

4.1 A model that follows the circuit’s dynamics

spice2rnm detects the level dependence during characterisation and builds a model whose dynamics are *scheduled against measured large-signal behaviour* — including the circuit’s rise/fall timing asymmetry, which it measures at 95 ps and reproduces. At a 50% input duty the model matches the circuit’s output duty to 0.000 percentage points.

Transient equivalence	0.0769 against a 0.15 threshold	PASS
UVM-MS, DC section	51 checks, 0 failed	
UVM-MS, duty section	15 checks, 0 failed	
worst duty error	0.314 pp of a 0.500 pp tolerance	

4.2 Verified by function, not by formula

The generated environment for this block contains *no AC section* — by measurement, not by omission. A small-signal sweep linearises the circuit at one operating point, and a comparator in normal operation is never near that regime; checked that way, a demonstrably accurate model fails every point by more than 55 dB, telling you nothing about the model and everything about the mismatch between the check and the block.

Instead, the environment checks what a duty-cycle corrector is *for*. The tool drives the circuit with clocks at multiple rates and input duties, records the output duty from SPICE, and holds the model to those goldens — a table spanning 20 percentage points of measured behaviour, so a model that ignored its input could not reproduce it.

5 Case study 3 — Phase interpolator

A thermometer-coded phase interpolator has essentially no amplitude transfer at all — there is no frequency response worth fitting, and a voltage-domain comparison scores noise. Conventional flows stop here.

spice2rnm recognises the block’s function and switches strategy: it measures phase per code, generates a delay-line model, and emits a *phase* verification environment — one measured golden per control code, no DC or AC section, because neither describes edge placement.

Codes measured	9 of 9
Total traversal	−500.1 ps, monotonic
Mean step (1 LSB)	−62.51 ps
Worst DNL	−100.5 ps (−1.61 LSB)
UVM-MS phase section	9 checks, 0 failed
worst	3.26 ps of a 6.25 ps tolerance

The tolerance is a tenth of an LSB, at the model’s own quantisation floor — tight enough to fail on a real regression, never on rounding. The DNL figure is a property of the circuit, reported faithfully; the model’s job is to reproduce it, and it does.

6 Case study 4 — Phase-locked loop

The first three are blocks. This one is a *loop*, and that changes what a model has to get right. A small error in any block does not stay small: the loop integrates it into a different operating point, so a model that is individually plausible everywhere can still settle the system somewhere the circuit does not.

It is also three different characterisation problems in one netlist, and no single method covers them. The pipeline classifies each block from its topology and routes it accordingly:

Block	Why the default does not fit	Route taken
ro_vco	oscillates with its inputs at DC — its output is a frequency, not a gain	tuning curve, scored on rate
cpump	a complementary pair driven from independently controlled gates	one $I(V)$ table per control state
lpfilt	only R and C, so its state space is exact and needs no fitting	exact state space, read from topology

None of those is a transfer function, and a conventional flow that fits one to each would be fitting the wrong quantity three times.

6.1 Checked as a system, against the transistors

Per-block scores are evidence about each block alone. The question a loop asks is different — do these models behave like the circuit once wired together — and the co-simulation answers it directly. The design’s own testbench runs twice, with only its DPI bridge functions rewritten: once reading the generated models, once calling the real analog through ngspice. Thresholds, expressions and verdict are byte-identical in both, so the two boundary logs compare directly.

Blocks modelled, three routes	3 composed, 2 more emitted
Per-block equivalence	5 of 5 pass
System check vs transistors	4 boundary signals, 0 failed
vout (control voltage)	0.224 mV
aout (VCO output)	0.4% of threshold crossings
dra (internal probe)	1.12 mV
Both loops lock at	400.00 MHz
lock points	1.8502 V vs 1.8499 V
Model half vs transistor half	2.12 s vs 66.1 s

The control voltage is the number to read. It is where the loop stores its state, so a model that settles anywhere else disagrees here first — and the two agree to a quarter of a millivolt while reaching the same frequency. Run to 20 μ s, where both halves settle rather than being caught mid-transient, the agreement improves to 0.135 mV.

6.2 Where the block works is measured, not assumed

The two output-buffer inverters are clock-path blocks: their function is edge placement, and an rms voltage error is close to blind to it. Both pass an rms comparison comfortably while their worst-instant error is 99.9% of the full swing, because a model whose edge is displaced in time sits at one rail while the reference sits at the other. The pipeline classifies them from the measured DC curve — 62% and 61% of swept points within 5% of a rail is a switching characteristic, not a smooth response — and scores them on duty cycle instead.

The frequency it scores them at is measured. Where a block operates is a property of the closed loop and exists only once the digital side is attached, exactly like the operating range the golden probe already measures. The probe therefore measures each net’s switching rate beside its voltage band: the two inverter inputs return 399.8 and 399.7 MHz, the 400 MHz this PLL locks to. Scored there, they pass at 0.276 and 0.307 pp of duty against a 0.5 pp bar.

The form they are re-emitted in has no delay line, so it cannot carry the measured rise/fall skew — 0.589 and 0.672 pp of duty at that clock. Both blocks measure below their own floor, so the skew partly cancels rather than adding. That makes the floor an upper bound on what the form costs, not margin to spend, and the pipeline states it before scoring rather than leaving a pass to be read as unqualified.

That number is not one a language model can supply. Asked five times about the same inverter with the same measurements, the classifier proposed clock rates of 2 MHz, 765 MHz, 300 MHz, 10 GHz and 15 GHz; asked about the second stage, it proposed 10 GHz. The lowest is two hundred times below where the buffer runs and the highest thirty-seven times above what its own model can follow. The measurement returns the same answer every run, and gives the two stages answers agreeing to 0.03%, which no classifier run did. Intent from the model, physics from the circuit, applied to a number that looked like intent and was not.

What the classifier is asked for, it has been steady on: buffer, clock stimulus, duty metric, on every run of both stages. That is the judgement gated on a measurement it can check itself against. What moved is the one number it was in no position to know.

The verdict names the metric it came from. Asking for the classifier is not the same as getting it. Every duty figure above comes from a run with a key present and the classifier consulted. The same demonstration was then run deliberately *without* one, to see what a reader would be shown if it declined: the chirp is kept, and these same two blocks pass an rms comparison at 0.025 against a 0.15 bar — a green result from a measurement that cannot see the property in question. The two transcripts are otherwise alike, and the command line is

identical, so the run records which figure each verdict was reached by and the demonstration reports it from that record. A flag requesting a check is not evidence the check happened.

7 Results at a glance

Case study	Environment	Checks	Failed
Filter	DC + AC	51 + 19	0
Duty corrector	DC + duty	51 + 15	0
Phase interpolator	phase per code	9	0
Phase-locked loop	co-simulation vs transistors	5 + 4	0

The first three case studies — characterisation, model generation, both simulators, and every generated environment — run end to end in under 16 minutes on a laptop. The PLL is a further 15 minutes, most of it the ring oscillator’s tuning curve.

All four are one command, and `demo_4_pll.sh` runs this one. It differs from the others in what it needs rather than in how it is run: being the analog half of a co-simulation, it needs the other half — the RTL, the DPI bridge, and a *shared* libngspice, which is a different ngspice build from the binary the first three use. It checks for each of those before doing any work and names what is missing, so a machine without them gets a skip that says which piece and where it comes from, rather than a failure twelve minutes into a run. spice2rnm’s outputs are simulator-agnostic — plain IEEE 1800 SystemVerilog, tied to no vendor — and to keep every result in this paper reproducible by anyone, all of it was produced with universally available open-source simulators: **ngspice** for characterisation and the reference transients, and **xezim** for both the transient equivalence co-simulation and the generated UVM-MS environments. No commercial licence was used anywhere in this document. The generated artifacts themselves — the models, the complete environments, and the evidence files, exactly as the tool wrote them — are published for inspection at

<https://github.com/vrajeshprakhya/spice2rnm-demos>

under `showcase/`, stamped with the tool version and run summary that produced them. What the repository holds:

```
spice2rnm-demos/
demo_1_lpf.sh  demo_2_dcc.sh  demo_3_pi.sh  run_all.sh
                the three case studies, runnable end to end
harness/      8 measurement scripts the demos call
showcase/     the generated artifacts, byte-for-byte
lpf2/         the baseline case, 14 files:
  rc_lpf2_rnm.sv          the model, plain real ports
  rc_lpf2_rnm_wreal.sv    the same model, wreal ports
  rc_lpf2_ms_types_pkg.sv the generated nettype
  rc_lpf2_dms.sv          net <-> variable, interconnect ports
  rc_lpf2_bridge*.sv      MS bridge and its driving core
  rc_lpf2_proxy_pkg.sv    agent <-> bridge API
  rc_lpf2_ms_pkg.sv       agent, and the SPICE goldens
  rc_lpf2_ms_scoreboard.svh the checks and tolerances
  rc_lpf2_ms_tb/test.svh  environment and test
  top_rc_lpf2_ms.sv       nets, DUT, bridge, test
  run_rc_lpf2_ms.sh       compiles and runs it
  result.json            fit, verdict, warnings
dcc2/         same shape, 15 files: no AC section, adds
                duty goldens in dcc_ms_pkg.sv and the run
                log pipeline.txt
pi/           same shape, 11 files: phase-per-code
```

	goldens, no DC or AC section
house_lib/	4 files, NOT generated: ng_ams_pkg.sv declaring nettype real ng_anet, and two hand-written models using it
lpf2_house/	13 files: lpf2 again, joined to house_lib -- no types package, and rc_lpf2_rnm_ng_anet.sv in place of the wreal boundary
STAMP	date, tool version, and run summary that produced the artifacts above
refresh.sh	regenerates everything; the only way this directory is ever updated
whitepaper/	this document, source and PDF

And what it deliberately does not hold: the spice2rnm tool itself (the product is licensed separately; this repository is its evidence), and the build products of the runs — characterisation scratch, compiled simulator objects, DPI shims. `refresh.sh` filters those out, so the showcase is what a user would keep, not everything a run produces.

8 Why a green testbench is worth believing

Generated verification has a credibility problem: if the same tool writes the model and the test, what stops it from marking its own homework? Three design rules.

Goldens are measurements, never the model. Every golden value in every generated environment is a SPICE measurement of your circuit. Checking a model against the equations it implements would pass unconditionally; spice2rnm never does it.

Every environment is proven able to fail. The tool injects faults into the generated model and confirms the verdict flips — and the detection floor is *measured*, not asserted. For the duty-cycle corrector’s environment:

Injected fault	Detected by
comparator offset, 20 mV	DC and duty sections
dynamics slowed 10×	duty section (DC is structurally blind)
dynamics slowed 50×	duty section (DC is structurally blind)

The middle column is the point: settled-DC checks — the whole of what many hand-written environments test — cannot see *any* error in a block’s dynamics. The function-matched sections exist because of that blindness, and the fault sweep is re-run against the shipped model so the detection claims stay current.

Sensitivity is pinned from both sides. A check that fails everything is as useless as one that passes everything, so fault injection also confirms that errors *within* tolerance do not trip the environment.

9 Functional specifications as input

A netlist says what a circuit *is*. It never says what it is *for*, what rate it must run at, or how close is close enough. Hand those to the tool as a plain-language document — a datasheet excerpt, a requirements page — and it reads out the facts that shape verification:

From the specification	Used for
operating clock range	where the environment drives the block
accepted input duty range	the stimulus sweep
duty accuracy requirement	the pass/fail tolerance
block function	which environment is generated

On the duty-cycle corrector, the specification-driven run checks 35–65% input duty at the specified clock rates against the specification’s own 1.0 pp requirement: 15 checks, 0 failed, worst 0.428 pp. **PASS**

The specification is cross-checked, never trusted. Every extracted fact is validated against what the circuit was actually measured to do, and the measurement wins. When the two disagree, that disagreement is surfaced as a finding:

```
SPEC CONFLICT: the spec says this block operates up to 4e+08 Hz, but
the fitted model is only valid below 1.195e+08 Hz -- 3.3x lower.
```

That is a coverage gap between a requirement and a characterisation, caught at model-generation time instead of at silicon correlation — and it is the kind of finding no amount of manual model review produces, because no one document contains both halves of it.

10 Joining an existing model library

A team with real-number models already has conventions — which user-defined nettype analog values travel on, which package declares it, what discipline their model ports use. Generated files that import their own freshly-invented nettype do not join that library; they sit beside it, each net scheme blind to the other, and the integration cost lands on the customer.

Point `--style-from` at the library and the output conforms: the environment’s analog nets use *their* nettype, its files import *their* package, no types package of the tool’s own is emitted, and the generated run script compiles their package file. A library whose models use `wreal` ports gets the `wreal` wrapper without being asked. Their timescale is reported but deliberately not adopted — generated delays are computed femtosecond values, and timescale is per compilation unit, so the directives interoperate unchanged.

Reading a house style is a judgement, so it is read and then verified. A library may declare several nettypes, only one of which is the living standard — and what says so is often a comment. Measured on a library that declares two, where the deprecated one happens to sort first alphabetically:

mechanical scan	picks the deprecated nettype, and warns
model reading	picks the live one, quoting the deprecation notice
verification	re-checks the choice against the named file

The model’s stated grounds, verbatim from that run: *“line 15 marks `aaa_legacy_wire` deprecated (“kept only so ancient testbenches still compile. Do NOT use in new code”), ... both models declare all analog ports as `acme_ewire`, confirming this is the live convention.”* Nothing in that reading is taken on trust: the named nettype, package and file are checked against the file before use, and a claim the file does not support leaves the mechanical answer standing, out loud. Where the tool has a *measurement* — a count of which discipline the library’s ports actually use — the count wins outright and a disagreement is recorded rather than adopted.

The result is an environment that runs on the house net, and it is published rather than described. The showcase carries the same filter twice: `lpf2/` generated normally, and `lpf2_house/` generated against `house_lib/` — a small hand-written library standing in for a customer’s own. Diffing the two directories is the whole argument:

	lpf2/	lpf2_house/
types package	emitted	not emitted
nets	rc_lpf2_wire	ng_anet
imports	the generated one	ng_ams_pkg
run script	compiles it	compiles the house package
model ports	plain real	ng_anet, and the environment instantiates it
the model	byte-identical	
the scoreboard	byte-identical	

Conformance changes the boundary and nothing else — and the published environment runs from where it sits, against the library beside it: 51 DC and 19 AC checks, 0 failed, exactly the default environment’s result.

Note what the environment instantiates in that second column. When the model is delivered behind a port boundary, the generated testbench drives *that* module rather than the core inside it, and the boundary’s file joins the compile list — so what the evidence covers is what ships, not a component of it. Measured separately, a library resolving its net by averaging rather than summing conforms and passes the same way, and a library using **wreal** ports gets the **wreal** wrapper instead. Without **--style-from**, or against a directory with nothing recognisable in it, nothing changes — and the run says so.

11 AI where it helps, physics where it decides

spice2rnm uses language models at four points: proposing block boundaries in the device graph, classifying what a block is for, reading functional specifications, and reading an existing library’s conventions (Section 10). All four follow one rule: **intent from the model, physics from the measurement**. Every proposal is validated against the measured circuit before it is used, and a proposal that fails validation is discarded — the run continues on measured defaults, and says so. Without an API key, every AI feature declines cleanly and the pipeline runs entirely on measurement.

Which of those two happened is recorded in the run’s own result, not only in its log. A verdict carries the metric it was reached by, so a report read months later distinguishes a check that ran from one that declined and fell back — without which every green result would have to be taken on trust that the feature requested was the feature exercised.

12 Scope and requirements

- **Inputs:** SPICE netlists, flat or with subcircuits; optional plain-language functional specification; optionally an existing RNM library to take conventions from.
- **Outputs:** SystemVerilog real-number models, with the port discipline of your choice — plain **real** (portable anywhere), **wreal**, or your library’s own nettype; UVM-MS 1.0 environments; equivalence reports with archived waveforms.
- **Toolchain:** a SPICE simulator for characterisation, and a digital simulator capable of running real number models — with UVM support for the generated environments. The generated models and environments are standard SystemVerilog and carry no vendor dependence, so either may be the one your flow already uses.
- **Block-level scope:** single-input single-output blocks and compositions of them, including current-mode blocks sharing a node. Hierarchical composition trades some accuracy for structure; the equivalence report states the composed figure separately from the per-block figures, so the trade is visible, not hidden.

© 2026 Veylon Systems. spice2rnm is a Veylon Systems product. Every figure in this document was produced by the shipped tool from its bundled demonstration circuits, most recently on 2026-09-16. The demonstration suite at <https://github.com/vrajeshprakhya/spice2rnm-demos> regenerates them in one command.

Veylon Systems